Lecture 7: Few-Shot Learning, Continual/Online Learning, and Test-Time Adaptation (January 29)

This lecture bridges the few-shot meta-learning framework developed in Lectures 1–6 with the broader challenges of **continual learning**, **online learning**, and **test-time adaptation (TTA)**.

Recap: The Few-Shot Learning Setup

The meta-learning framework operates over a distribution of **tasks**. Each task T consists of:

$$T = \{\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}\}$$

where $\mathcal{D}_{\text{train}} = (X_{\text{train}}, Y_{\text{train}})$ is the support set and $\mathcal{D}_{\text{test}} = (X_{\text{test}}, Y_{\text{test}})$ is the query set. The model is trained over many such tasks $T_0, \ldots, T_{N-1}$ and evaluated on held-out tasks.

The two algorithms covered so far:

- **Matching Networks** (metric-based): learn an embedding space; classify by attention-weighted comparison at test time.
- **MAML** (optimization-based): learn an initialization θ; adapt via gradient descent on the support set at test time.

Continual and Online Learning

The lecture draws a sharp distinction between the meta-learning setup and two related but different paradigms:

Continual / Online Learning Setup

Tasks arrive **sequentially**: first $T_0, \ldots, T_{N-r}$ (training phase), then $T_{N+1}, \ldots, T_{N+M}$ (deployment phase).
During the training phase, the model learns parameters θ from the initial batch of tasks. During deployment:

- The model **uses** θ to make predictions on new tasks.
- It receives **feedback** (labels, rewards, or other signals) on its predictions.
- It must **update** to incorporate this new information.

Key difference from meta-learning: In few-shot meta-learning, all tasks during training have complete support and query sets. In continual/online learning, the model encounters tasks in a stream and must update on the fly.

The Forgetting Problem

Online learning focuses on doing well on *new* tasks and learning from them — it does not prioritize retaining performance on old tasks.

Continual learning adds a harder constraint: the system must also **retain performance on old tasks** while learning new ones. This is the catastrophic forgetting problem (Lecture 1), now cast in the task-sequential setting.

Test-Time Adaptation (TTA): Introduction

The lecture introduces a new problem setting that connects meta-learning to distribution shift:

Test-Time Adaptation (TTA)

Given:

- A model trained on tasks $T_0, \ldots, T_{N-1}$ (standard meta-training).
- At test time, we receive $\mathcal{D}_{\text{test}} = X_{\text{test}}$ **without labels**.

The key constraint: we have **no or sparse** labels at test time (Y_{train} is unavailable or very limited for the new task).

Goal: Adapt f_θ at test time using only the unlabeled test data X_{test}, possibly combined with the structure of $\mathcal{D}_{\text{train}}$.

This is a realistic scenario: in deployment, we often encounter distribution shift but cannot obtain labels for the shifted data quickly enough. TTA asks whether we can still adapt the model using only the test inputs themselves.

The lecture previews that TTA will require **unsupervised adaptation signals** — losses that do not require ground-truth labels but still provide useful gradient information for adapting the model. The next lectures will develop these signals in detail.

Lecture 8: MAML Variants, Model-Based Meta-Learning, and LMs as Meta-Learners (February 2)

This lecture extends MAML with efficient approximations, introduces the third family of meta-learning (model-based), draws a deep connection between meta-learning and language models, and formalizes test-time adaptation architectures.

MAML Recap and the Cost of Second-Order Gradients

Restating the full MAML algorithm:

Inner loop (for each task T_k): starting from $\theta' = \theta$, for $s = 1, \ldots, S$:

$$\theta' \leftarrow \theta' - \varepsilon \, \nabla_{\theta'} \mathcal{L}^I\big(f_{\theta'}(X_{\text{train}}), Y_{\text{train}}\big) \tag{1}$$

Outer loop (meta-update):

$$\theta \leftarrow \theta - \eta \sum_k \nabla_\theta \, \nabla_{\theta'} \, \mathcal{L}^O\big(f_{\theta'}(X_{\text{test}}), Y_{\text{test}}\big) \tag{2}$$

The outer gradient $\nabla_\theta \mathcal{L}^O$ requires differentiating through the inner loop optimization. Since $\theta' = \theta - \varepsilon \nabla_\theta \mathcal{L}^I(\theta)$, this involves **second-order derivatives** (Hessian-vector products). These are expensive to compute and store: for a model with $|\theta|$ parameters, the Hessian is $|\theta| \times |\theta|$.

The computational profile: the inner loop cost scales with ε (step size) and S (number of steps). Higher-order gradients scale with ε^2. This motivates Newton-style methods ($[\nabla]^{-1}$) and, more practically, first-order approximations.

First-Order MAML and Reptile

Reptile (First-Order MAML)

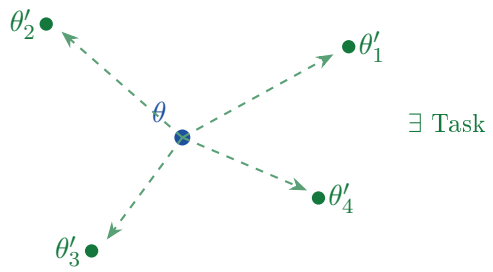
Drop all second-order terms. Instead of differentiating through the inner loop, simply move the initialization θ toward the adapted parameters θ'.

Single task:

$$\theta \leftarrow \theta - \eta \, (\theta - \theta') \tag{3}$$

Batch of tasks:

$$\boxed{\theta \leftarrow \theta - \eta \sum_k (\theta - \theta'_k)} \tag{4}$$



Geometric intuition: Each $(\theta - \theta'_k)$ is a vector pointing from the adapted parameters back to the current initialization. Summing these and subtracting moves θ toward the "centroid" of all task-specific optima. Reptile finds an initialization that is close to all task solutions without ever computing second-order gradients.

Why it works: Reptile implicitly approximates the full MAML gradient. When expanded to second order, the Reptile update and the MAML update share the same leading terms — the difference is only in higher-order corrections that have diminishing impact in practice.

Standard ML vs. Meta-Learning: Training Loop Comparison

Side-by-Side Comparison

Standard ML (batch-based):

- Data: $(X_{B_0}, Y_{B_0}), (X_{B_1}, Y_{B_1}), \ldots, (X_{B_n}, Y_{B_n})$ — batches from one dataset
- Per batch: compute $\mathcal{L}_{B_i}$, compute $\Delta\theta_{B_i}$
- One pass through all batches = one epoch

Meta-Learning (task-based):

- Data: $T_0, T_1, \ldots, T_n$ — each task has its own $\mathcal{D}^{T_i}_{\text{train}}, \mathcal{D}^{T_i}_{\text{test}}$
- Per task: compute $\mathcal{L}(\mathcal{D}^k_{\text{test}} \mid \mathcal{D}^k_{\text{train}})$, compute $\Delta\theta_k$
- Aggregate: $\Delta\theta_B = \theta - \eta \sum_{k=0}^{n} \nabla_\theta \mathcal{L}(\mathcal{D}^k_{\text{test}} \mid \mathcal{D}^k_{\text{train}})$
- One pass through a "batch of tasks" = one episode

The structural parallel is exact: where standard ML iterates over *data batches*, meta-learning iterates over *task batches*. The key difference is that each "sample" in meta-learning is itself a learning problem.

Model-Based Meta-Learning: The Third Family

The lecture introduces the third major family of meta-learning, completing the taxonomy:

Family	**Representative**
Metric-based	Matching Networks
Optimization-based	MAML
Model-based	**CNP, LSTM meta-learner**

Model-Based Meta-Learning

Instead of learning a metric or an initialization, learn a model F that **directly maps** the support set and a test input to a prediction:

$$\boxed{F(x, \mathcal{D}_{\text{train}}) \to y} \tag{5}$$

The entire support set $\mathcal{D}_{\text{train}}$ is an *input* to the model. There is no explicit inner loop (unlike MAML) and no explicit distance computation (unlike Matching Networks). The model must implicitly learn how to extract task-relevant information from the support set.

Conditional Neural Processes (CNP)

CNP Architecture

Step 1: Encode each support pair. Each (x_i, y_i) from $\mathcal{D}_{\text{train}}$ is passed through an encoder g_ϕ:

$$(x_i, y_i) \xrightarrow{g_\phi} h_i, \qquad i = 0, 1, \ldots, N-1$$

where the labels $y_i \in \{0, 1, \ldots\}$ indicate class membership.

Step 2: Aggregate per class. For each class c_j, compute the class representation by averaging:

$$r_{c_j} = \frac{1}{K} \sum_{i:y_i=c_j} h_i^{c_j}$$

This creates a "prototype" for each class — a summary of what class c_j looks like.

Step 3: Predict. Concatenate the class representations $r_{c_0}, r_{c_1}, \ldots$ with the test point embedding $m = g(x)$ and feed through a decoder f_D:

$$\hat{y} \leftarrow f_D([m,\ r_{c_0},\ r_{c_1},\ \ldots])$$

The per-class averaging connects CNP to **Prototypical Networks** (Snell et al., 2017): both compute class centroids in embedding space. The difference is that CNP uses a learned decoder rather than nearest-centroid classification.

LSTM Meta-Learner (Hochreiter et al., 2001)

One of the earliest meta-learning approaches, predating modern terminology:

Learning to Learn with Gradient Descent by Gradient Descent

Architecture: An LSTM f_θ processes the support set **sequentially**:

$$x_0, x_1, x_2, \ldots, x_{N-1} \xrightarrow{\text{LSTM } f_\theta} \hat{y}_0, \hat{y}_1, \ldots, \hat{y}_{N-1}$$

The targets are shifted: the output at position i predicts y_i (the label for the *current* input, informed by all previous (x, y) pairs seen so far).

How it works: The LSTM's hidden state accumulates knowledge about the task as it processes more (x, y) pairs. By the time it reaches the test points, its hidden state encodes a "learned model" for this specific task.

At test time: Continue feeding test inputs x_{test} through the same LSTM. The hidden state (now encoding the task from the support set) produces predictions $\hat{y}_{\text{test}}$.

Training loss:

$$\mathcal{L} = \mathcal{L}^{\text{train}} + \mathcal{L}^{\text{test}}$$

optimized end-to-end with $\nabla_\theta \mathcal{L}$.

This is a pure model-based approach: the LSTM *is* the learning algorithm. There are no gradient steps at test time — the LSTM's recurrent computation implicitly performs the adaptation.

Relation to Language Models: LLMs as Meta-Learners

LLMs are Meta-Learners with Expanding Support Sets

In autoregressive language modeling, the input-output structure at each position is:

$$X : w_0 \qquad Y : w_1 \tag{6}$$
$$X : w_0, w_1 \qquad Y : w_2 \tag{7}$$
$$X : w_0, w_1, w_2 \qquad Y : w_3 \tag{8}$$

The "support set" $\mathcal{D}_{\text{train}}$ **continuously expands** as the context grows. Each new token becomes part of the context for predicting the next token.

This is structurally identical to meta-learning: the LLM is a model-based meta-learner where the "task" is defined by the prompt, and the "support set" is the growing context window. In-context learning (ICL) is meta-learning at inference time.

Semi-Supervised Meta-Learning

The lecture also briefly introduces a setting where the support set contains both labeled and unlabeled data:

Semi-Supervised Meta-Learning

$\mathcal{D}_{\text{train}}$ is split:

- **Labeled portion:** $x_0, \ldots, x_k$ with labels $y_0, \ldots, y_k$
- **Unlabeled portion:** $x_{k+1}, \ldots, x_{n-1}$ with labels **missing**

Goal: Use the unlabeled data to improve adaptation of f_θ beyond what the labeled data alone provides. This is domain adaptation *within* the meta-learning framework.

Test-Time Adaptation: The Dual-Head Architecture

The lecture formalizes the TTA framework introduced in Lecture 7:

Dual-Head TTA Architecture

The model has three components:

- **Shared encoder:** $g(x) \to h$ — maps input to a hidden representation.
- **Prediction head:** $f^p(h) \to \hat{y}$ — maps representation to class prediction.
- **Reconstruction head:** $f^r(h) \to \hat{x}$ — maps representation back to input space.

During training (both losses available):

$$\mathcal{L} = \underbrace{\mathcal{L}_p(y, \hat{y})}_{\text{prediction loss}} + \underbrace{\mathcal{L}_r(x, \hat{x})}_{\text{reconstruction loss}}$$

During test-time adaptation (no labels):
Only $\mathcal{L}_r(x, \hat{x}) = \|x - \hat{x}\|$ is available. But adapting the *shared encoder* g with the reconstruction loss implicitly improves the prediction head f^p as well, because both heads share the same representation.

Why Reconstruction Helps Prediction

The reconstruction head acts as a **regularizer** and **unsupervised adaptation signal**. If the test distribution has shifted (e.g., different image styles, new domains), the reconstruction loss detects this shift — $\|x - \hat{x}\|$ increases for out-of-distribution data. Minimizing reconstruction loss on test data adapts the encoder to the new distribution, and this adaptation transfers to the prediction head through the shared representation.

This connects to **energy-based models**: the reconstruction error can be viewed as an energy function, and adaptation minimizes the energy on the test distribution.

Lecture 9: Test-Time Adaptation via Self-Critique and Fast Weight Learning (February 5)

This lecture deepens the TTA framework with learned critique functions and establishes a fundamental connection between gradient-based adaptation and the attention mechanism.

TTA Signals: Beyond Reconstruction

The dual-head architecture from Lecture 8 used reconstruction loss as the unsupervised signal. The lecture identifies three complementary signals for test-time adaptation:

1. **Reconstruction loss:** $\mathcal{L}_r = \|x - \hat{x}\|$ — adapt the encoder to reconstruct test inputs well.
2. **Entropy minimization:** The prediction distribution $\hat{y}$ over classes should be *confident* (peaky), not uniform. High entropy means the model is uncertain — minimize:

$$\mathcal{L}_{\text{ent}} = -\sum_c \hat{y}_c \log \hat{y}_c$$

3. **Pseudo-labels:** Use the model's own predictions as targets and self-train. If the model predicts class k with high confidence, treat that as a "label" and fine-tune on it.

All three signals use only X_{test} — no ground-truth labels required. The challenge is that these signals are noisy: entropy minimization can reinforce wrong predictions, and pseudo-labels can cause error propagation.

Learning to Learn TTA via Self-Critique

The lecture presents a meta-learning approach to *learn* the TTA loss function itself:

L2L via Self-Critique — Full Algorithm

While not converged:

1. **Sample a task** T
2. **Standard MAML inner loop (supervised):**

$$\begin{aligned}
&\theta' \leftarrow \theta \quad \{\text{clone}\}\\
&\text{for } s = 0 \dots S:\\
&\qquad \mathcal{L}^I = \mathcal{L}\big(f_{\theta'}(X_{\text{tr}}), Y_{\text{tr}}\big)\\
&\qquad \theta' \leftarrow \theta' - \varepsilon \nabla_{\theta'} \mathcal{L}^I
\end{aligned}$$

3. **Unsupervised adaptation (via learned critique):**

$$\begin{aligned}
&\theta'' \leftarrow \theta'\\
&\text{for } k = 0 \dots K:\\
&\qquad F = \Big[f_{\theta''}(X_{\text{test}}),\ \ g_\phi(X_{\text{test}}, X_{\text{train}}),\ \ X_{\text{train}},\ \ X_{\text{test}}\Big]\\
&\qquad \mathcal{L}^C = C_w(F) \quad \{\text{critique loss}\}\\
&\qquad \theta'' \leftarrow \theta'' - \gamma \nabla_{\theta''} \mathcal{L}^C
\end{aligned}$$

4. **Outer loss and meta-update:**

$$\begin{aligned}
\mathcal{L}^O &= \mathcal{L}\big(f_{\theta''}(X_{\text{test}}), Y_{\text{test}}\big)\\
\theta &\leftarrow \theta - \eta \nabla_\theta \mathcal{L}^O \quad \{\text{MAML outer step}\}\\
(w, \phi) &\leftarrow (w, \phi) - \alpha \nabla_{(w,\phi)} \mathcal{L}^O
\end{aligned}$$

What is the critique network C_w? It is a **learned loss function**. Its input F contains:

- $f_{\theta''}(X_{\text{test}})$: the model's current test embeddings
- $g_\phi(X_{\text{test}}, X_{\text{train}})$: cross-similarity features between test and train
- Raw features $X_{\text{train}}, X_{\text{test}}$

C_w is trained to *mimic* the kind of loss that, when used for adaptation, leads to good outer-loop performance. It learns to approximate entropy, reconstruction, and other useful unsupervised signals — but in an end-to-end, task-adaptive way.

Why Learn the Critique?

Hand-designed TTA losses (reconstruction, entropy) are generic. By *meta-learning* the critique C_w, we learn a loss function that is optimized specifically for enabling good test-time adaptation. The outer MAML loop provides the training signal: "did this critique loss actually lead to better predictions on the query set?"

MAML for Language Models: Fast Weight Learning

The lecture now applies the MAML idea to **language models**, revealing a deep connection between gradient-based adaptation and attention.

Setup: Language Model Adaptation

A language model predicts the next token:

$$p_\theta(x_n \mid x_0, \ldots, x_{n-1})$$

The output at each position is a probability vector p_{n+1} over the vocabulary V:

$$p_{n+1} = [p_{n+1}^{(0)},\ p_{n+1}^{(1)},\ \ldots,\ p_{n+1}^{(|V|)}]^\top$$

The loss at position i is the negative log-probability of the correct next token k:

$$\ell_i = -\log p_i^{(k)} \quad \Longrightarrow \quad \nabla_{p_i^{(k)}} \ell_i = \frac{1}{p_i^{(k)}} \tag{9}$$

Inner Loop: Gradient Accumulation

Following the MAML/fast-weight paradigm, we adapt a parameter θ' at each position t by accumulating gradients from all preceding positions:

$$\theta' \leftarrow \theta \quad \text{(clone)} \tag{10}$$

$$\theta' \leftarrow \theta' - \sum_{i \le t} \nabla_\theta \ell_i \qquad \text{for } t = 0, \ldots, n-1 \tag{11}$$

This is "fine-tuning" the model on its own context — each token's prediction error is used to adjust the model for better predictions on subsequent tokens.

The Key Derivation: Gradient Descent ≡ Attention

Assume the prediction head is a **linear layer**: $f_\phi(h) = h_i \cdot \theta \to p_i$, where:

- $h_i \in \mathbb{R}^{1\times d}$ is the hidden state at position i
- $\theta \in \mathbb{R}^{d\times|V|}$ is the weight matrix
- $p_i \in \mathbb{R}^{|V|}$ is the output probability vector

The gradient of ℓ_i with respect to θ uses the chain rule:

$$\nabla_\theta \ell_i(p_i) = \underbrace{\nabla_{p_i}\ell_i(p_i)}_{1/p_i^{(k)}} \cdot \underbrace{\nabla_\theta p_i}_{h_i^\top} \tag{12}$$

since $p_i = h_i\theta$ implies $\nabla_\theta(h_i\theta) = h_i^\top$.

Define the gradient signal: $g_i = 1/p_i^{(k)}$ (a scalar encoding how "surprised" the model was at position i).

Substituting into the adapted parameters:

$$\theta' = \theta - \alpha \sum_{i\le t} h_i^\top g_i \tag{13}$$

Now compute the adapted prediction at position t:

$$\begin{aligned} p_t' &= h_t \cdot \theta' \\ &= h_t \left(\theta - \alpha \sum_{i\le t} h_i^\top g_i\right) \\ &= \underbrace{h_t\theta}_{p_t} - \alpha \sum_{i\le t} \underbrace{h_t h_i^\top}_{\text{attention}} \cdot g_i \end{aligned} \tag{14}$$

The Fundamental Equivalence

$$\boxed{p_t' = p_t - \alpha \sum_{i\le t} (h_t^\top h_i) \cdot g_i} \tag{15}$$

The correction term $\sum_{i\le t}(h_t^\top h_i) \cdot g_i$ is a **dot-product attention mechanism**:

- **Query:** h_t (current hidden state)
- **Keys:** h_i (past hidden states)
- **Values:** $g_i = 1/p_i^{(k)}$ (gradient signals / prediction errors)

Gradient-based adaptation of a linear layer is mathematically equivalent to an attention layer.

This is a profound result. It means that when a transformer uses attention over past context, it is implicitly performing a form of gradient descent on its own predictions. The "fast weights" that are updated via gradient descent are equivalent to the key-value memory that attention mechanisms maintain.

Fast Weight Learning: Summary of Insights

Four Key Takeaways

1. **Adapt a fast-weight layer (MLP):** Apply MAML-style gradient descent to the final linear layer of an LM. This is "MAML on LM."
2. **Also done at training time:** The fast-weight updates can be incorporated during training, not just at test time.
3. **Adaptation $\equiv$ another attention layer:** The derived update rule is mathematically identical to a cross-attention layer attending over the context.
4. **Use pseudo-labels for TTA:** Since the gradient signal g_i depends on the model's own predictions, this extends naturally to test-time adaptation without ground-truth labels.

Lecture 10: Critique Networks, Meta-Networks, and Applications (February 9)

This lecture completes the meta-learning framework with learned critique functions for TTA, introduces meta-networks that generalize MAML, and demonstrates a financial application.

Critique Networks for TTA: Full Formalization

Expanding on the self-critique framework from Lecture 9:

Critique Network C_w

The critique function takes as input a feature vector F and outputs a scalar loss for adaptation:

$$\mathcal{L}^C = C_w(F), \qquad F = \left\{ f_\theta(X_{\text{test}}),\ \ g(X_{\text{test}}, X_{\text{train}}),\ \ \theta^2 \right\} \tag{16}$$

The inputs to C_w include:

- $f_\theta(X_{\text{test}})$: test point embeddings (how the model currently represents test data)
- $g(X_{\text{test}}, X_{\text{train}})$: cross-features capturing the relationship between test and train distributions
- θ^2: model parameters (optional, enables the critique to reason about the model's state)

The function g acts as a reconstruction / "embedding" function — it captures how well the test data fits within the representation space learned from the training data. The critique C_w is designed to **mimic** unsupervised losses (entropy, reconstruction) that do not require labels, but in a way that is optimized end-to-end for downstream prediction performance.

Meta-Networks: Generalizing MAML

MAML uses a specific adaptation mechanism: gradient descent. The lecture asks: *why gradient descent?* What if we replace it with a **learned function**?

In MAML, the inner loop produces adapted parameters:

$$\theta' = \theta - \alpha \nabla_\theta \mathcal{L}^I(\mathcal{D}_{\text{train}}) \tag{17}$$

The question: can we replace $\theta - \alpha \nabla_\theta \mathcal{L}^I$ with a **learned mapping** m?

Meta-Network Variants

Three progressively more general formulations:

(a) Learned optimizer:

$$\theta' = m_z(\nabla_\theta \mathcal{L}(\mathcal{D}_{\text{train}})) \tag{18}$$

A neural network m_z takes the per-example gradients $\ell_0(x_0, y_0), \ell_1(x_1, y_1), \ldots$ and outputs adapted parameters. This **replaces gradient descent** with a learned optimization rule.

(b) Test-point-specific adaptation via attention:

$$\theta^j_{\text{test}} = \text{Softmax}\big(x_j^{\text{test}} \cdot X_{\text{train}}\big) \cdot \begin{bmatrix} \theta'_0 \\ \vdots \\ \theta'_K \end{bmatrix} \tag{19}$$

For each test point x_j^{test}, compute attention weights over the training points and take a **weighted combination of task-specific adapted parameters**. Different test points get different effective models.

(c) Learned similarity for adaptation:

$$\theta^j_{\text{test}} = \text{Softmax}\big(g_w(x_j^{\text{test}}) \cdot g_u(X_{\text{train}})\big) \cdot \begin{bmatrix} \theta'_0 \\ \vdots \\ \theta'_K \end{bmatrix} \tag{20}$$

Same as (b) but with **learned encoders** g_w, g_u instead of raw dot products. This is the most flexible variant.

Hyper-Networks

Meta-networks are an instance of **hyper-networks** — networks whose weights are produced or modified by another (sub-)network. In variant (a), the sub-network m_z directly produces the adapted weights. In variants (b) and (c), an attention mechanism over training examples dynamically computes test-point-specific weights. This is strictly more expressive than MAML's fixed gradient descent rule.

The outer loss and meta-update remain the same as MAML:

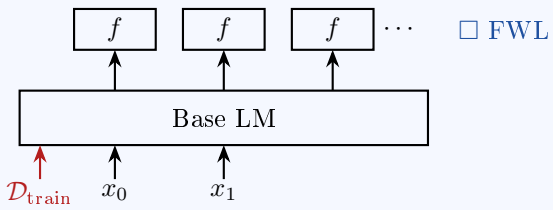
$$\mathcal{L}^O = \mathcal{L}\big(f_{\theta'}(X_{\text{test}}), Y_{\text{test}}\big), \qquad \theta \leftarrow \theta - \eta\, \nabla_\theta \mathcal{L}^O \tag{21}$$

FWL Applied to Language Models

Connecting Fast Weight Learning (Lecture 9) back to the meta-network perspective:

FWL as MAML Applied to LMs

A language model has a base model producing hidden states, with a fast-weight (FW) layer on top:



The inner loop adapts only the FW layer:

$$\theta' \leftarrow \theta - \varepsilon \sum_{i \le t} \nabla \ell_i \tag{22}$$

As shown in Lecture 9, this is equivalent to an attention layer. The adapted prediction can also be written using a moving average of gradients:

$$\theta' \leftarrow \text{maj}(\nabla_i) \tag{23}$$

where maj denotes a (possibly learned) aggregation of the per-position gradient signals.

Application: Financial Prediction (Numerai-Style)

The lecture concludes with a practical application of meta-learning to quantitative finance.

Financial Meta-Learning Setup

Data structure: 50 assets (stocks) observed over n days. Each asset j on day d has a feature vector $x_d^j \in \mathbb{R}^m$ (technical indicators, fundamentals, etc.) and a return r_d^j.

The data forms a matrix:

$$\begin{pmatrix} x_0^0 & x_0^1 & \cdots & x_0^{50} \\ x_1^0 & x_1^1 & \cdots & x_1^{50} \\ \vdots & & & \vdots \\ x_n^0 & x_n^1 & \cdots & x_n^{50} \end{pmatrix} \leftarrow \mathcal{D}_{\text{train}} \text{ (returns)}$$

Output: Strategy positions p_{n+1} for the next day, where for each asset i:

$$p_k^{(i)} \in \{-1, 0, 1\}$$

- $+1$: long (buy) the top k assets
- -1: short (sell) the bottom k assets
- 0: no position for the remaining assets

Portfolio allocation: Given total capital A, the strategy allocates:

- $A/3$ equally across long positions (each gets $A/3k$)
- $A/2$ equally across short positions
- Remaining in cash

The decision of which assets to go long/short is exactly a **few-shot classification problem**: given recent market data (the support set), classify each asset as "buy," "sell," or "hold." The meta-learning framework applies directly — train on historical periods, adapt to new market regimes at test time.

The Complete Meta-Learning Taxonomy

With all four lectures, the taxonomy is now complete:

Family	What is Learned	Adaptation	Representatives
Metric-based	Embedding space / similarity function	Compare test to support via attention	Matching Networks, Prototypical Networks
Optimization-based	Good initialization θ	Gradient descent on support set	MAML, FOMAML, Reptile
Model-based	A model that reads the support set	Forward pass through the model	CNP, LSTM meta-learner, Meta-Networks
TTA / Critique-based	Unsupervised adaptation loss	Self-supervised signals at test time	Dual-head TTA, L2L via self-critique, FWL

Summary: Lectures 7–10

Lecture 7 (Jan 29): Few-shot vs. continual/online learning. The forgetting problem. TTA introduction: adapting without labels at test time.

↓

Lecture 8 (Feb 2): MAML → FOMAML/Reptile. Model-based meta-learning: CNP, LSTM meta-learner. LLMs as meta-learners with expanding $\mathcal{D}_{\text{train}}$. Dual-head TTA architecture.

↓

Lecture 9 (Feb 5): TTA signals (reconstruction, entropy, pseudo-labels). L2L via self-critique: meta-learning the TTA loss. Fast Weight Learning: gradient descent on a linear layer ≡ attention.

↓

Lecture 10 (Feb 9): Critique networks. Meta-networks: replacing GD with learned optimizers and hyper-networks. FWL on LMs. Financial application of meta-learning.

These four lectures extend the meta-learning framework in three directions:

1. **Efficient optimization:** FOMAML and Reptile show that dropping second-order gradients barely hurts performance while dramatically reducing computation. The geometric insight — moving θ toward the average of task-specific optima — provides clean intuition.

2. **Model-based approaches:** CNP and LSTM meta-learners show that adaptation can be implicit (the model *reads* the support set) rather than explicit (gradient steps). The connection to LLMs is deep: in-context learning *is* model-based meta-learning.

3. **Test-time adaptation:** When labels are unavailable at test time, unsupervised signals (reconstruction, entropy, pseudo-labels) can drive adaptation. The self-critique framework meta-learns which signals work best. The Fast Weight Learning derivation reveals that **gradient-based adaptation of a linear layer is mathematically equivalent to attention** — unifying the optimization and attention perspectives.

The deepest insight across these lectures is the **equivalence between gradient descent and attention**. This means the three families of meta-learning are not as separate as they appear:

- Metric-based methods use **attention** to compare.
- Optimization-based methods use **gradient descent** to adapt.
- But gradient descent on a linear layer *is* attention.

The families are different perspectives on the same underlying computation.